

Model pretnji i bezbednosna analiza

Sistem *Oblak* – platforma za izvršavanje korisničkog koda
u izolovanim Firecracker mikro-VM okruženjima

Bezbednost informacionih sistema

May 24, 2026

Contents

1	Uvod i obim analize	2
1.1	Ciljevi dokumenta	2
1.2	Tehnologije u sistemu	2
2	Arhitektura sistema i granice poverenja	2
2.1	Visok nivo arhitekture	2
2.2	Granice poverenja (trust boundaries)	3
2.3	Dijagram toka podataka (DFD)	3
3	Imovina i ciljevi zaštite	4
4	STRIDE analiza pretnji	5
4.1	Spoofing – lažno predstavljanje	5
4.2	Tampering – neovlašćeno menjanje	6
4.3	Repudiation – poricanje	7
4.4	Information disclosure – otkrivanje informacija	7
4.5	Denial of service – uskraćivanje usluge	8
4.6	Elevation of privilege – eskalacija privilegija	9
4.7	Sumarna STRIDE tabela	11
5	Bezbednosni zahtevi izvedeni iz pretnji	12
6	Detaljna analiza izvršavanja koda u VM-u	13
6.1	Slojevi zaštite (defense in depth)	13
6.2	Statičke heuristike u <code>CodeAnalysisService</code>	14
6.3	Tok izvršavanja, korak po korak	14

6.4	Ono što bi trebalo dodatno uraditi	15
7	Mehanizam revizije (audit log)	15
7.1	Predloženi model	15
7.2	Tačke u kodu gde treba dodati ulaze	16
7.3	Otpornost na izmenu	16
8	Razvojni proces	16
8.1	Verzionisanje i <i>commit</i> istorija	16
8.2	Code review	17
8.3	Statička analiza koda samog projekta	17
9	Otvorene stavke – sumarno	18
10	Zaključak	19
A	Mapiranje koda na bezbednosne mehanizme	19
B	Reference	19

1 Uvod i obim analize

Ovaj dokument predstavlja model pretnji i bezbednosnu analizu sistema *Oblak* – platforme koja korisnicima omogućava da preko CLI alata (`cdk`) otpreme ZIP arhivu sa Python projektom i pokrenu je u izolovanom Firecracker mikro-virtuelnom okruženju na strani servera. Sistem je u akademskom kontekstu razvijen kao prototip i analiza je sprovedena nad implementacijom dostupnom u repozitorijumu *oblak* (Spring Boot 4 / Java 17) i `cdk-cli` (Python 3, biblioteka `click`).

Analiza se sprovodi nad celovitim sistemom, sa posebnim fokusom na izvršavanje korisničkog koda unutar virtuelne mašine – izolovanost procesa, ograničavanje resursa i sprečavanje bekstva iz VM-a. Bezbednosni zahtevi su izvedeni iz STRIDE klasifikacije pretnji, a za svaku identifikovanu pretnju navedena je implementirana mera zaštite ili, ako mera nije implementirana, opisana je kao otvorena stavka sa preporukom za budući rad.

1.1 Ciljevi dokumenta

- Definirati arhitekturu sistema i bezbednosne granice (*trust boundaries*).
- Pomoću STRIDE metodologije identifikovati pretnje na nivou sistema i na nivou izvršavanja koda u VM-u.
- Definirati bezbednosne zahteve i mapirati ih na implementirane mehanizme.
- Navesti otvorene stavke – pretnje koje su prepoznate ali još nisu adresirane.
- Pratiti razvoj kroz *commit* istoriju i *code review* praksu, uz preporuke za statičku analizu koda.

1.2 Tehnologije u sistemu

- **Klijent** (`cdk-cli`): Python 3, `click`, `requests`; lokalna konfiguracija u `~/.cdk/config.json`.
- **Server** (*oblak*): Spring Boot 4, Spring Security, Spring Data JPA, JWT.
- **Skladište artefakata**: MinIO (S3-kompatibilan), `bucket=projects`.
- **Baza podataka**: PostgreSQL.
- **Izvršno okruženje**: Firecracker mikro-VM sa Ubuntu 24.04 rootfs i prilagođenim `vmlinux-6.1.155` kernelom; korisnički kod montiran kao `ext4` disk `/dev/vdb`.
- **Statička analiza otpremljenog koda**: `bandit` (Python), `clamscan` (ClamAV), `pip-audit`.

2 Arhitektura sistema i granice poverenja

2.1 Visok nivo arhitekture

Sistem ima tri sloja: klijent (CLI), aplikacioni server (Spring Boot), i izvršno okruženje (Firecracker VM). Komunikacija između klijenta i servera ide preko HTTP REST API-ja

sa JWT autentifikacijom. Server orkestrira upload, validaciju, analizu koda, izgradnju `ext4 filesystem image`-a i kasnije pokretanje VM-a sa tim image-om kao drugim diskom.

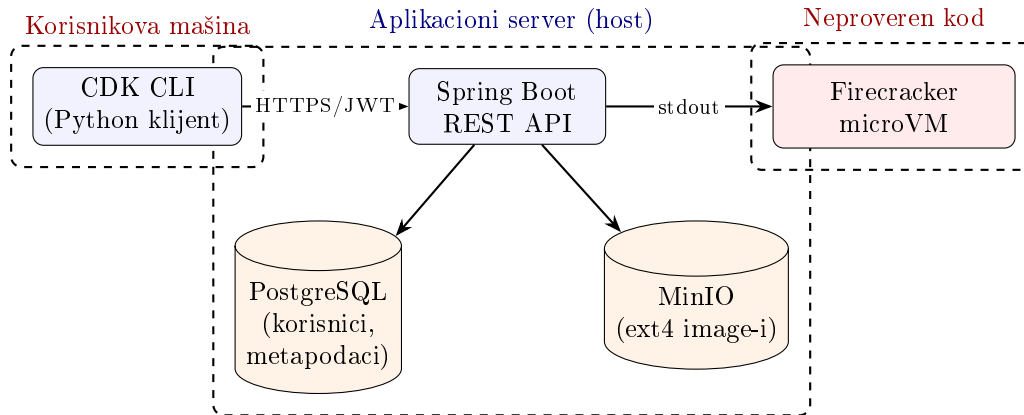


Figure 1: Komponente sistema sa naznačenim granicama poverenja.

2.2 Granice poverenja (trust boundaries)

Identifikovane su tri značajne granice na koje treba obratiti posebnu pažnju:

- TB1: Klijent → Server.** Granica između nepoverljivog klijenta i aplikativnog sloja. Svaki podatak koji uđe preko ove granice (kredencijali, ZIP fajlovi, `projectId`) tretira se kao potencijalno zlonameran i mora proći validaciju.
- TB2: Server → skladišta.** Granica između aplikacionog procesa i pomoćnih servisa (PostgreSQL, MinIO). Pristup je ograničen kredencijalima i mrežnom konfiguracijom.
- TB3: Host → Firecracker gost.** *Najkritičnija granica.* Sve što se dešava unutar VM-a treba smatrati neproverenim kodom – VM *ne sme* da pristupi host fajl-sistemu, mreži ka unutrašnjim servisima (PostgreSQL, MinIO, sam server), niti da prepozna druge VM-ove.

2.3 Dijagram toka podataka (DFD)

Tabela 1 mapira komponente sistema na standardne DFD elemente koji se koriste u STRIDE analizi (procesi, skladišta podataka, eksterni entiteti, tokovi podataka).

Table 1: Elementi dijagrama toka podataka.

Oznaka	Tip	Element
EE1	Eksterni entitet	Korisnik (operator CLI alata)
P1	Proces	CDK CLI (klijentska aplikacija)
P2	Proces	<code>AuthController</code> / <code>AuthService</code>
P3	Proces	<code>UploadController</code> / <code>UploadService</code>
P4	Proces	<code>ExecutionController</code> / <code>ExecutionService</code>
P5	Proces	<code>FirecrackerOrchestrator</code> (pokretanje VM-a)
P6	Proces	Firecracker microVM (korisnički proces)
DS1	Skladište podataka	PostgreSQL (<code>app_user</code> , <code>refresh_token</code> , <code>project_metadata</code>)
DS2	Skladište podataka	MinIO bucket <code>projects</code>
DS3	Skladište podataka	Host fajl-sistem (<code>temp/</code> , <code>/tmp/firecracker-*</code>)
DS4	Skladište podataka	Klijentska konfiguracija <code>~/.cdk/config.json</code>

3 Imovina i ciljevi zaštite

Pre nego što se identifikuju pretnje, korisno je jasno definisati šta se štiti. Imovinu sistema čine:

- A1: Korisnički kredencijali.** Korisnička imena, lozinke (hash-ovane), JWT *access* i *refresh* tokeni.
- A2: Korisnički kod (projektne arhive).** Otpremljene ZIP arhive i njihovi `ext4` image-i u MinIO-u; metapodaci o projektima.
- A3: Integritet host sistema.** Sam aplikacioni server, njegov fajl-sistem, mreža, procesi.
- A4: Dostupnost servisa.** Kapacitet servera da prihvati upload i obradi izvršavanje za sve korisnike.
- A5: Privatnost izvršavanja.** Garancija da se izvršavanje jednog korisnika ne meša sa izvršavanjem drugog (izolacija između VM-ova).
- A6: Tragovi (audit trail).** Logovi koji omogućavaju reviziju ko je šta uradio i kada.

Bezbednosni ciljevi prate klasičnu CIA trijadu proširenu autentičnošću i mogućnošću revizije:

- **Poverljivost:** kredencijali i sadržaj projekata ne smeju biti dostupni neovlašćenim licima.
- **Integritet:** ZIP arhive i `ext4` image-i ne smeju se neopaženo menjati između upload-a i izvršavanja.
- **Dostupnost:** nijedan korisnik ne sme svojim akcijama (*zip bomb*, beskonačna petlja, ...) zaglaviti sistem za druge.
- **Autentičnost:** svaka akcija mora biti vezana za autentifikovanog korisnika.

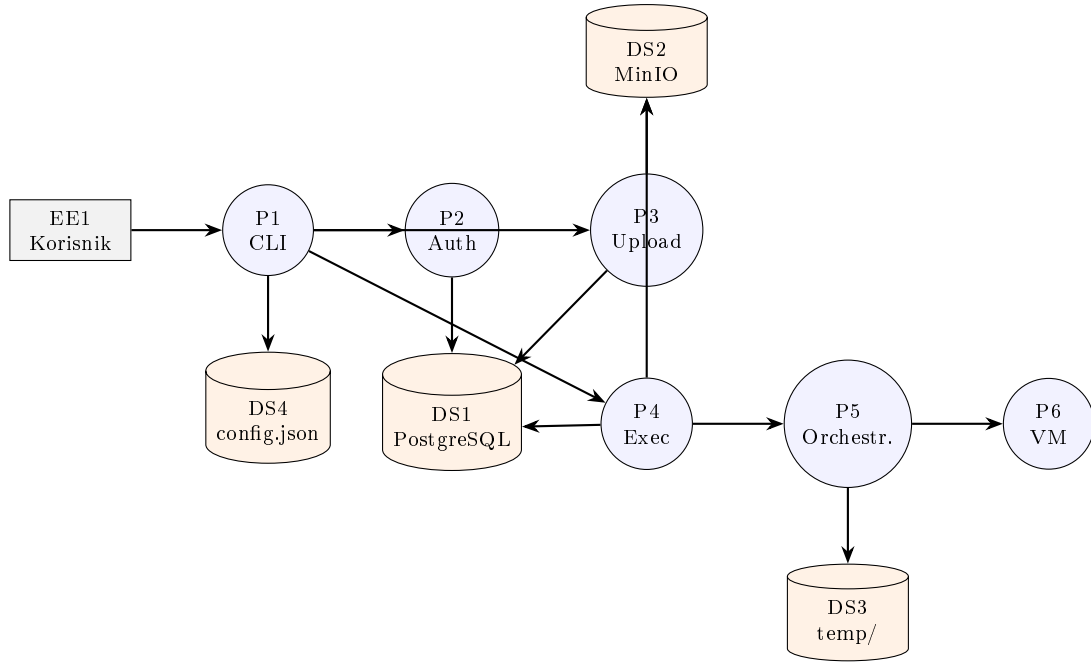


Figure 2: DFD: tok podataka kroz sistem. Granice poverenja prelaze se između korisničke mašine i servera (P1→P2/P3/P4) i između host-a i VM-a (P5→P6).

- **Neporecivost (audit):** značajni događaji moraju biti zabeleženi tako da se kasnije mogu revidirati.

4 STRIDE analiza pretnji

STRIDE je akronim za šest kategorija pretnji: **S**poofing (lažno predstavljanje), **T**ampering (neovlašćeno menjanje), **R**epudiation (poricanje), **I**nformation disclosure (otkrivanje informacija), **D**enial of service (uskraćivanje usluge), i **E**levation of privilege (eskalacija privilegija). Sledi sistematična analiza svake kategorije nad arhitekturom sistema.

4.1 Spoofing – lažno predstavljanje

S1: Lažno predstavljanje kao drugi korisnik preko ukradenog JWT-a. Ako napadač dođe do tuđeg *access* tokena (npr. pristupom `~/.cdk/config.json`), može da otpremi i pokreće projekte u ime žrtve.

Implementirana mera: JWT tokeni imaju važenje 24h (`jwt.expiration-ms=86400000`); *refresh* tokeni se skladište u bazi sa mogućnošću *revoke*-ovanja; *cdk* klijent postavlja `chmod 0600` na config fajl (što ograničava čitanje na vlasnika POSIX naloga). Token se prenosi kao *Bearer* u *Authorization* header-u.

Otvorena stavka: 24h *access* token je dug; kraći (npr. 15 min) sa automatskim *refresh*-om bio bi bezbedniji. Takođe, trenutno *access* token nije ograničen *jti*-om u bazi, pa se ne

može revoke-ovati pojedinačno – moraće se sačekati da istekne. Skraćivanje *access* tokena i uvođenje *jti* blacklist-e su preporuka za nastavak rada.

S2: Spoofing samog servera (napadač podmeće lažan OBLAK_SERVER_URL). Klijent po default-u koristi `http://localhost:8080`, ali se URL može promeniti komandom `cdk config -server`. Bez TLS-a, MITM napadač na mreži može čitati i menjati saobraćaj.

Otvorena stavka: TLS trenutno nije konfigurisan (Spring Boot sluša običan HTTP). Za produkcionu upotrebu obavezno: konfigurisati HTTPS sa validnim sertifikatom, eventualno *certificate pinning* na strani klijenta, i odbijati HTTP *schema* u `server_url`.

S3: Lažno predstavljanje pri registraciji (preuzimanje tuđeg username-a). Napadač koji prvi registruje popularno korisničko ime može ga čuvati za kasnije.

Implementirana mera: `userRepository.findByUsername(...)` sprovodi `UNIQUE` constraint; ponovljena registracija vraća `409 CONFLICT`. Ovo je dovoljno za prototip; identitet ne može biti dupliciran.

4.2 Tampering – neovlašćeno menjanje

T1: Zip slip – ZIP arhiva sa putanjama tipa `../../../../etc/passwd`. Ako server ekstrahuje ZIP bez provere, fajlovi mogu završiti van predviđenog direktorijuma i prepisati osetljive fajlove host-a.

Implementirana mera: `ZipValidation.validateEntryPath` odbija unose koji počinju sa `/`, sadrže `../../../../`, završavaju sa `/. . .`, sadrže nul-bajt, ili su Windows apsolutne putanje. Dodatno, `ZipExtractionService` koristi `targetPath.normalize()` i proverava `startsWith(tempDir)` pre upisivanja.

T2: Path traversal kroz ime fajla iz `ProjectMetadata.workingDirectory`. `ROOT_FILE` i `PROJECT_ROOT` se prosleđuju kernelu kao *boot args*. Ako ime fajla sadrži razmake, navodnike ili \$-znakove, mogla bi se desiti *argument injection* na nivou `agent-runner.sh`-a.

Otvorena stavka: u `FirecrackerOrchestrator.buildBootArgs` vrednosti se direktno konkatenuiraju u *boot args* string. Treba dodati strogu validaciju (whitelist znakova: alfanumerički, taraba, tačka, donja crta, kosa crta), ili eskejpovanje. U trenutnoj implementaciji vrednosti dolaze iz `ProjectMetadata`-e koju popunjava sam server (ne korisnik direktno), pa je rizik smanjen, ali ne i isključen.

T3: Tampering nad ext4 image-om u MinIO-u. Napadač sa pristupom MinIO bucket-u može zameniti image projekta i ubaciti zlonameran kod.

Implementirana mera: MinIO je u trenutnoj konfiguraciji dostupan isključivo aplikacionom serveru preko lokalne mreže/kontejnera.

Otvorena stavka: ne čuva se kriptografski hash (npr. SHA-256) ext4 image-a u `ProjectMetadata`-i. Preporuka: pri `markCompleted`-u sačuvati hash, a pre pokretanja VM-a u `ExecutionService` verifikovati da preuzeti image još uvek odgovara tom hashu.

T4: Tampering nad lokalnim CLI config fajlom. Drugi proces na korisnikovoj mašini može izmeniti `server_url` ili `token`.

Implementirana mera: `chmod 0600`. Dodatne mere su izvan dometa jednostavnog CLI-a.

4.3 Repudiation – poricanje

R1: Korisnik može da porekne otpremanje zlonamernog projekta. Bez tragova, teško je dokazati da je određeni korisnik otpremio određenu arhivu u određeno vreme.

Trenutno stanje: `ProjectMetadata` sadrži `owner_id` (referenca na `User`-a), ali *ne* vremenske oznake otpremanja, niti su sačuvani rezultati skeniranja, niti hash arhive.

Otvorena stavka – prioritarna. Treba implementirati zaseban `AuditLog` entitet (videti sekciju 7) koji čuva: korisnika, akciju (`REGISTER/LOGIN/UPLOAD/EXECUTE/REJECTED`), vremensku oznaku, IP adresu, `projectId` kad je primenljivo, rezultat (`SUCCESS/DENIED/ERROR`). Logovi treba da budu *append-only*.

R2: Operater servera može izmeniti logove. Lokalni administrator host-a ima pun pristup logovima i može ih izmeniti.

Otvorena stavka: za prototip prihvatljivo; za produkciju – centralizovani logging (Syslog, ELK, ili pisanje hash-eva logova u poseban append-only servis poput `auditd`-a) sa potpisivanjem ulaza.

4.4 Information disclosure – otkrivanje informacija

I1: Otkrivanje postojanja tuđeg projekta. Ako `POST /api/execute/{id}` vrati različite kodove grešaka za "projekat ne postoji" naspram "nemate dozvolu", napadač može učiniti *enumeration* UUID-ova.

Implementirana mera: u `ExecutionService.execute` se eksplicitno vraća `404 NOT_FOUND` i kad projekat postoji ali nije u vlasništvu korisnika (komentar u kodu to izričito navodi). UUID-ovi su 128-bitni i nepogodni za brute-force.

I2: Slabe lozinke. Ako sistem dozvoli slabu lozinku, napadač može brute-force-ovati nalog.

Implementirana mera: `PasswordPolicyService` zahteva minimum 8 karaktera, veliko slovo, malo slovo, cifru, specijalan karakter, zabranjuje 4 uzastopna ista znaka, zabranjuje da lozinka sadrži username, i proverava listu čestih lozinki iz `passwords/common-passwords.txt`. Lozinke se hash-uju BCrypt-om.

I3: Hardkodovan JWT *secret* u `application.properties`. Trenutno je u repozitorijumu zakucan tajni ključ:

```
1 jwt.secret=1UDIQ3QYqdc0o26Sp6xHT1h5lo12Elvy3MykCBKoyRZxxtQa31K7
2 TxEPi7JIXe9cH3WNoKyFIo331AxpKQ64wg==
```

Listing 1: Trenutna konfiguracija u `application.properties` (skraćeno)

Otvorena stavka – prioritetna. Svako ko ima pristup repozitorijumu može da kreira validne JWT tokene za bilo kog korisnika. Treba premestiti `jwt.secret` u *environment varijablu* (Spring već omogućava `${JWT_SECRET}`), generisati ga na svakom okruženju zasebno, i *ne* commit-ovati u repozitorijum. Isto važi za `spring.datasource.password=admin123`.

I4: Otkrivanje internih putanja kroz stdout VM-a. `FirecrackerOrchestrator` vraća pun stdout/stderr `Firecracker` procesa, koji uključuje host putanje (`/tmp/firecracker-...`). Ovaj sadržaj klijent vidi u `ExecutionResult`.

Otvorena stavka: klijentu treba vraćati samo `programStdout` i `programStderr` (output korisničkog programa), a `Firecracker`-ov sopstveni stdout/stderr (sa host putanjama) zadržati samo serverski u logu, ne u API odgovoru.

I5: Otkrivanje šeme baze kroz H2 konzolu. `SecurityConfig` dozvoljava `/h2-console/**` bez autentifikacije.

Otvorena stavka: aplikacija koristi PostgreSQL u `application.properties`, pa H2 konzola verovatno nije aktivna, ali *permitAll* pravilo treba ukloniti jer može biti zaboravljeno ako se H2 ikad uključi u test-profilu.

4.5 Denial of service – uskraćivanje usluge

D1: Zip bomb (*decompression bomb*). Mali ZIP koji se ekstrahuje u gigabajte podataka može popuniti disk host-a.

Implementirana mera: `ZipValidation` u toku *stream*-ovanog čitanja broji `totalExtractedSize` i prekida čim pređe `oblast.zip.max-total-extracted-size-bytes` (default 100 MB).

Takođe ograničava broj fajlova na `obлак.zip.max-files` (default 1000) i odbija ugnježdene ZIP fajlove. Upload sam je ograničen na 20 MB (`spring.servlet.multipart.max-file-size`).

D2: Beskonačna petlja ili *CPU/memory hog* u korisničkom kodu. Korisnički program ne mora da se završi; može iscrpiti CPU, RAM, ili napuniti gostov disk.

Implementirane mere:

- `FirecrackerConfigBuilder.buildMachineConfig` fiksira `vcpu_count=1` i `mem_size_mib=1024`, tako da VM ne može da preuzme više od jednog vCPU-a i 1 GiB RAM-a.
- `FirecrackerOrchestrator` pokreće `process.waitFor(maxExecutionTimeSeconds, ...)` (default 120s) i poziva `destroyForcibly()` ako VM ne završi u tom roku.
- U `agent-runner.sh`-u dodatno postoji `timeout 10s python3 ... - guest-side hard cap` koji onemogućava da se sam Python proces izvršava duže od 10 sekundi.
- ext4 image projekta je fiksne (mala) veličine – maksimum (`projectSize + imageExtraSizeBytes`) – pa zapisivanje u gostov fajl-sistem brzo nailazi na `ENOSPC`.

Komentar: obratiti pažnju na nekonzistenciju: server dozvoljava 120s, a guest ima limit od 10s. Guest-side limit nadvladava i bolji je odbrambeni sloj, ali bi trebalo eksplicitno dokumentovati ovaj kontrast u kodu (komentar uz `timeout 10s`).

D3: Iscrpljivanje host-ovih privremenih direktorijuma. Svaki upload piše u zajednički `temp/` direktorijum.

Implementirana mera: `UploadService.upload` ima `finally` blok koji čisti `projectImage` i poziva `zipExtractionService.cleanTempDirectory()`.

Otvorena stavka: `temp/` je deljen između svih konkurentnih upload-a – ako dva korisnika rade upload paralelno, `cleanTempDirectory` jednog može obrisati fajlove drugog. Treba koristiti `Files.createTempDirectory("upload-")` po zahtevu (kao što već radi `FirecrackerOrchestrator`), umesto fiksnog `temp/`.

D4: Spam registracija i *credential stuffing*. Endpoint `/auth/login` i `/auth/register` nemaju *rate limiting*.

Otvorena stavka: dodati limiter (npr. `bucket4j` sa Spring filter-om) – ograničiti broj pokušaja po IP/username u jedinici vremena. Dodati *exponential backoff* ili *lockout* posle više neuspešnih prijava.

4.6 Elevation of privilege – eskalacija privilegija

E1: Bekstvo iz Firecracker VM-a na host. Ovo je glavna i centralna pretnja celog sistema – da zlonameran korisnički kod pobegne iz VM-a i izvrši kod na host-u.

Implementirane mere – više slojeva odbrane (defense in depth):

- **Firecracker po definiciji** koristi KVM, minimalan *device model* i prepravljen *boot* mehanizam; njegov *attack surface* je dramatično manji od QEMU-a. Koristi se za AWS Lambda i Fargate baš zbog kvaliteta izolacije.
- **Bez mreže:** `FirecrackerConfigBuilder` ne konfigurise `network-interfaces`, što znači da gost *nema* mrežu; ne može da kontaktira PostgreSQL, MinIO, ni Internet. Ovo automatski eliminiše celu klasu pretnji.
- **Read-only rootfs:** `drive rootfs` je konfigurisan sa `is_root_device=true`, a `is_read_only=false` – *ovo treba ispraviti na true* (otvorena stavka, videti dalje), čime bi se onemogućilo da korisnički kod menja sistemske fajlove unutar VM-a.
- **Project disk kao podatak, ne kao izvršni sistem:** korisnički kod je na `/dev/vdb` (ext4), montiran u `/mnt/project` – ne može da prebriše rootfs.
- **Statička analiza pre izvršavanja:** `CodeAnalysisService` pre upload-a izvršava `bandit` (Python security linter), `clamscan` (ClamAV antivirus), `pip-audit` (vulnerable dependencies), i lokalna *regex*-pravila za eksplicitno zabranjene konstrukcije (`eval`, `exec`, `os.system`, `subprocess`, `socket`, `ctypes`, ...). $\text{Score} \geq 4$ odbija upload.
- **Fiksiran RAM i vCPU** (1 GiB / 1 vCPU) ograničavaju eventualne side-channel i resource-exhaustion napade.

Otvorene stavke – prioritetno:

1. **`is_read_only=true` za rootfs.** Trenutno je `false`; trebalo bi staviti na `true` da bi se sistemski fajlovi gosta zaštitili od izmena unutar jedne sesije.
2. **Pokretanje Firecracker-a pod ne-root korisnikom sa `seccomp` filterima.** Firecracker po default-u radi sa *seccomp* filterima koji ograničavaju *syscall*-ove koje host-side proces sme da izvršava. Treba potvrditi da je *seccomp enabled* u trenutnoj instalaciji (Firecracker ima opciju `-seccomp-filter`, ne sme se pokretati sa `-no-seccomp`).
3. **`cgroups` ili `jailer`.** Firecracker dolazi sa pomoćnim alatom `jailer` koji izoluje proces unutar `chroot`-a, mrežnog *namespace*-a, i `cgroup`-a. Trenutna implementacija pokreće `firecracker` direktno preko `bash -lc`; trebalo bi uvesti `jailer` u `ProcessBuilder` komandu.
4. **Run-as ne-root nalog na host-u.** Treba osigurati da Spring Boot proces nije root i da ima dovoljne KVM privilegije samo preko `kvm` grupe (`/dev/kvm`).

E2: Eskalacija unutar Spring Boot procesa preko *argument injection*. `FirecrackerOrchestrator` pokreće `ProcessBuilder` sa argumentima `"bash", "-lc"` i `String.format(...)` stringom.

Komentar i otvorena stavka: putanje koje se prosleđuju u `format` dolaze iz `configBuilder.buildConfig` i `Files.createTempDirectory`, ne direktno iz korisničkog unosa, pa je rizik realno mali. Međutim, `bash -lc` sa `String.format` je antipattern – bolje je pozvati `firecracker` di-

rektno, preko `ProcessBuilder`-a sa eksplicitnim argumentima ("`-api-sock`", "`-config-file`", ...), bez shell-a.

E3: Pretrazivost privatnih projekata. Ako bi neki API endpoint zaboravio da proveriti vlasništvo (`metadata.getOwner()` naspram trenutnog `requesterUsername`), korisnik bi mogao da pokrene tuđe projekte.

Implementirana mera: `ExecutionService.execute` eksplicitno proverava vlasništvo i vraća 404 ako se ne podudara. `UploadController` koristi `SecurityContextHolder` da uzme username iz JWT-a, ne iz tela zahteva.

4.7 Sumarna STRIDE tabela

Table 2: Pregled STRIDE pretnji i statusa adresiranja.

ID	Pretnja	Rizik	Status
S1	Krađa JWT tokena	Srednji	Delom (TTL, refresh revoke); kraći access TTL otvoren
S2	MITM bez TLS-a	Visok	Otvorena stavka (HTTPS)
S3	Duplikat username	Nizak	Adresirano (UNIQUE)
T1	Zip slip	Visok	Adresirano (dvostruka provera)
T2	Argument injection (<i>boot args</i>)	Srednji	Delom (vrednosti iz servera); whitelist otvoren
T3	Tampering MinIO image-a	Srednji	Otvorena stavka (hash)
T4	Tampering CLI configa	Nizak	Adresirano (<code>chmod 0600</code>)
R1	Poricanje upload-a	Visok	Otvorena stavka (audit log)
R2	Operater menja logove	Nizak	Otvorena stavka (centralni log)
I1	Enumeracija projekata	Nizak	Adresirano (404 umesto 403)
I2	Slabe lozinke	Srednji	Adresirano (policy + BCrypt)
I3	Hardkodovan JWT secret	Visok	Otvorena stavka (env var)
I4	Host putanje u stdout-u	Nizak	Otvorena stavka

ID	Pretnja	Rizik	Status
I5	H2 konzola otvorena	Nizak	Otvorena stavka (ukloniti pravilo)
D1	Zip bomb	Visok	Adresirano (streaming + limiti)
D2	Beskonačna petlja / CPU hog	Visok	Adresirano (višeslojni timeout)
D3	Iscrpljivanje deljenog temp/	Srednji	Otvorena stavka (po-zahtev temp dir)
D4	Brute-force login	Srednji	Otvorena stavka (rate limit)
E1	Bekstvo iz VM-a	Visok	Adresirano (Firecracker bez mreže + analiza); jačanje otvoreno (jailer, read-only rootfs)
E2	bash -lc antipattern	Nizak	Otvorena stavka (direktan exec)
E3	Pokretanje tuđeg projekta	Visok	Adresirano (provera vlasništva)

5 Bezbednosni zahtevi izvedeni iz pretnji

Na osnovu identifikovanih pretnji, sledi konsolidovana lista bezbednosnih zahteva koje sistem treba da ispuni. Svaki zahtev je označen oznakom (SR-X), STRIDE kategorijom, i statusom (**ispunjen** ili **otvoreno**).

SR-1 (S): Sistem mora autentifikovati svaki zahtev ka `/api/**` preko JWT *Bearer* tokena. – **Ispunjeno** (`JwtAuthFilter`, `SecurityConfig`).

SR-2 (S, I): Lozinke moraju biti hash-ovane sa adaptivnom funkcijom. – **Ispunjeno** (`BCrypt`).

SR-3 (S, I): Lozinke moraju ispunjavati definisanu politiku složenosti. – **Ispunjeno** (`PasswordPolicyService`).

SR-4 (S, T): Komunikacija između klijenta i servera mora biti zaštićena TLS-om. – **Otvoreno** (HTTPS još nije konfigurisan).

SR-5 (T): Uploadovan ZIP fajl mora biti validiran protiv *zip slip*-a, *zip bomb*-e i prekomernog broja unosa. – **Ispunjeno** (`ZipValidation` + `ZipExtractionService`).

SR-6 (T, I): Integritet otpremljenih image-a u skladištu mora biti proverljiv. – **Otvoreno** (hash u metapodacima).

- SR-7 (R):** Sve relevantne akcije (autentifikacija, upload, izvršavanje, odbijanje) moraju biti zabeležene u *audit* log. – **Otvoreno** (videti sekciju 7).
- SR-8 (I):** Endpoint-i ne smeju otkrivati postojanje resursa korisnicima koji nemaju pravo pristupa. – **Ispunjeno** (404 umesto 403).
- SR-9 (I):** Tajni ključevi (JWT secret, DB password) ne smeju biti u repozitorijumu. – **Otvoreno** (trenutno u `application.properties`).
- SR-10 (D):** Server mora ograničiti veličinu upload-a, broj fajlova u arhivi i ukupnu veličinu nakon ekstrakcije. – **Ispunjeno**.
- SR-11 (D, E):** Izvršavanje korisničkog koda mora imati ograničeno vreme, CPU, RAM i prostor na disku. – **Ispunjeno** (višeslojni timeout + Firecracker `machine-config`).
- SR-12 (D):** Endpoint-i za autentifikaciju moraju imati *rate limit*. – **Otvoreno**.
- SR-13 (E):** Korisnički kod mora se izvršavati u izolovanom VM-u bez mrežnog pristupa host servisima. – **Ispunjeno** (Firecracker bez `network-interfaces`).
- SR-14 (E):** Sumnjiv kod (definisan *score*-om iznad 4) mora biti odbijen pre dolaska u VM. – **Ispunjeno** (`CodeAnalysisService`).
- SR-15 (E):** Rootfs unutar VM-a treba da bude *read-only*. – **Otvoreno** (trenutno `is_read_only=false`).
- SR-16 (E):** Firecracker proces treba pokretati pod ne-root nalogom i jačati jailer-om. – **Otvoreno**.
- SR-17 (T, E):** Argumenti za eksterne procese ne smeju se kombinovati kroz shell sa korisničkim podacima. – **Delom** (vrednosti uglavnom dolaze sa servera; bolja eksplicitna validacija ostaje otvorena).

6 Detaljna analiza izvršavanja koda u VM-u

Ova sekcija se posebno fokusira na bezbednosna razmatranja oko izvršavanja korisničkog koda – najosetljivijeg dela sistema, jer korisnik unosi proizvoljan kod koji će se izvršiti.

6.1 Slojevi zaštite (defense in depth)

Sistem implementira višeslojnu odbranu pri izvršavanju korisničkog koda:

1. **Sloj 1 – Validacija ulaza.** `ZipValidation` odbija loše formirane ili *path-traversal* ZIP arhive još pre ekstrakcije.
2. **Sloj 2 – Statička analiza.** `CodeAnalysisService` pokreće `bandit`, `clamscan`, `pip-audit`, i lokalna *regex* pravila. $\text{Score} \geq 4$ rezultira `MaliciousCodeException`-om i odbijanjem upload-a.
3. **Sloj 3 – Hardver/virtualizacija.** Firecracker pokreće KVM-baziran microVM sa minimalnim *device model*-om. Iako je hipervizor, dramatično smanjuje napadačku

površinu u odnosu na QEMU.

4. **Sloj 4 – Bez mreže.** VM nema `network-interfaces` u konfiguraciji, pa ne može da pristupi internim servisima (PostgreSQL, MinIO, host).
5. **Sloj 5 – Ograničeni resursi.** 1 vCPU, 1 GiB RAM, mali ext4 image.
6. **Sloj 6 – Vremenski limit (višeslojan).** Host strana: 120s na nivou Firecracker procesa; guest strana: `timeout 10s` u `agent-runner.sh`-u koji ubija sam Python proces.
7. **Sloj 7 – Kontrolisani *init*.** Umesto da Linux gost pokrene normalan `init`, kernelu se preko `boot args`-a daje `init=/usr/local/bin/agent-runner`. Ovaj skript radi mountovanje, pokretanje korisničkog koda, i `halt/poweroff` odmah po završetku. Korisnik nema `shell`, nema `cron`, nema `daemon-e`.

6.2 Statičke heuristike u CodeAnalysisService

`isMaliciousByRules` primenjuje sledeće ne-osetljive na velika/mala slova:

```
1 return lower.contains("rm -rf")
2     || lower.contains("os.system")
3     || lower.contains("subprocess")
4     || lower.contains("eval(")
5     || lower.contains("exec(")
6     || lower.contains("socket")
7     || lower.contains("fork")
8     || lower.contains("pty")
9     || lower.contains("ctypes");
```

Listing 2: Lokalna pravila za prepoznavanje sumnjivog koda

Važno zapažanje: ova pravila su *lo-false-positive* skup – prepoznajuće za očigledno zlonamernan kod, ali trivijalno zaobilazna (npr. `getattr(__import__("os"), "system")(...)` ili `eval(chr(101)+...)`). Ona služe kao *dodatak*, ne kao primarna odbrana – primarna odbrana je sama Firecracker izolacija. Treba ih dokumentovati upravo tako u kodu, da se ne stvori lažni osećaj sigurnosti.

6.3 Tok izvršavanja, korak po korak

Sledi precizan opis šta se dešava kada korisnik pozove `cdk run <projectId>`:

1. Klijent šalje `POST /api/execute/{projectId}` sa JWT-om.
2. `JwtAuthFilter` parsira token, postavlja `SecurityContext`.
3. `ExecutionController` izvlači `username` iz `SecurityContextHolder`-a (ne iz tela zahteva).

4. `ExecutionService` učitava `ProjectMetadata` iz baze. Ako `owner` ne odgovara, vraća 404.
5. `minioService.downloadProjectImage` preuzima `ext4` image iz MinIO-a u privremeni fajl `host-a`.
6. `FirecrackerConfigBuilder` kreira JSON konfiguraciju sa: `kernelom`, `rootfs-om`, `project diskom`, `vcpu=1`, `mem=1024 MiB`, `log/metrics` putanjama.
7. `FirecrackerOrchestrator` pokreće `firecracker -api-sock ... -config-file ...` preko `ProcessBuilder`.
8. `Firecracker` učitava `kernel`; `kernel` iznosi `boot args` u kojima je `init=/usr/local/bin/agent-runner`.
9. `agent-runner.sh` montira `/dev/vdb` na `/mnt/project`, ulazi u `WORKDIR`, pokreće `timeout 10s python3 $ROOT_FILE`.
10. Po završetku Python procesa, skript ispisuje markirane sekcije (`program_stdout_begin, ...`), poziva `sysrq/halt/poweroff`.
11. Host strana čeka `waitFor(120s)`; nakon završetka VM-a hvata `stdout`, parsira sekcije, vraća `ExecutionResult`.
12. U `finally`-ju briše privremeni `ext4` fajl i `config` fajl; `deleteIfExists`.

6.4 Ono što bi trebalo dodatno uraditi

- **Read-only rootfs.** Postaviti `is_read_only=true` za `rootfs` drive (trivijalna izmena u `FirecrackerConfigBuilder`).
- **Jailer.** Pokretati `jailer` umesto `firecracker`-a direktno; `jailer` radi `chroot`, `cgroups` i `namespace-e`.
- **Seccomp.** Eksplicitno proveriti da `Firecracker` startuje sa default `seccomp` filterima (`never -no-seccomp`). Ovo dokumentovati.
- **Izolovani temp direktorijum po izvršavanju.** `FirecrackerOrchestrator` već koristi `Files.createTempDirectory("firecracker-...")`, što je dobro. `UploadService` ne – treba uskladiti.
- **Logovanje rezultata analize.** `System.out.println` u `CodeAnalysisService` treba zameniti pravim SLF4J logger-om i sačuvati rezultate (uz vezu na `projectId`) u bazi za naknadnu reviziju.

7 Mehanizam revizije (audit log)

Audit log je trenutno nedovoljno razvijen aspekt sistema. Postoje `System.out.println` ispisi i Spring Boot-ov standardni log, ali *nema* strukturisanog *audit trail*-a vezanog za korisnika. Ovde se opisuje minimalan *audit* mehanizam koji bi trebalo implementirati.

7.1 Predloženi model


```

1  @Entity
2  public class AuditLog {
3      @Id @GeneratedValue
4      private Long id;
5
6      private Instant timestamp;
7      private String username;           // null ako akcija nije
                                         autentifikovana
8      private String ipAddress;
9      private String action;           // REGISTER, LOGIN, LOGOUT, UPLOAD,
                                         UPLOAD_REJECTED, EXECUTE,
                                         EXECUTE_DENIED
10
11     private String resourceId;        // npr. projectId, ili null
12     private String outcome;          // SUCCESS, FAILURE, DENIED
13     private String detail;           // opciono: kratak opis (npr. score,
                                         razlog)
14 }

```

Listing 3: Predlog AuditLog entiteta

7.2 Tačke u kodu gde treba dodati ulaze

- AuthService.register → REGISTER / SUCCESS ili REGISTER / FAILURE (sa razlogom: password_policy ili username_taken).
- AuthService.login → LOGIN / SUCCESS ili LOGIN / FAILURE.
- AuthService.logout → LOGOUT.
- UploadService.upload → UPLOAD / SUCCESS sa projectId, ili UPLOAD_REJECTED sa razlogom (zip_validation, malicious_code, score=X).
- ExecutionService.execute → EXECUTE / SUCCESS (sa exit code-om), ili EXECUTE / FAILURE (timeout itd.), ili EXECUTE_DENIED (kad vlasništvo ne podudara).

7.3 Otpornost na izmenu

Za prototip dovoljno je da AuditLog entitet bude u istoj PostgreSQL bazi, sa *insert-only* pravom za aplikacionog korisnika (ne može UPDATE ni DELETE). Dugoročno – replikacija u eksterni log servis (Loki, ELK) ili auditd sa potpisom hash-eva ulaza.

8 Razvojni proces

8.1 Verzionisanje i *commit* istorija

Razvoj je praćen kroz Git *commit* istoriju. Preporučeni format poruka:

```

1 <tip>(<opcionni-modul>): <kratak opis>
2
3 [opcionni du\v zi opis]

```

Listing 4: Konvencija *commit* poruka

Tipovi: `feat`, `fix`, `security`, `refactor`, `docs`, `test`, `chore`. Posebnu pažnju zaslužuju `security-tip` komitovi, koji u poruci treba da pomenu odgovarajući STRIDE ID iz tabele 2 (npr. `security(zip): T1 - jačanje provere putanja`).

8.2 Code review

Za prototip se preporučuje princip "*four eyes*" za bilo koju izmenu koja:

- dodaje ili menja autentifikaciju, autorizaciju ili kriptografiju,
- menja `ZipValidation`, `CodeAnalysisService` ili `FirecrackerConfigBuilder`,
- menja `Firecracker boot args`, `drive` konfiguraciju ili kernel,
- dodaje novi `ProcessBuilder` poziv,
- uvodi novi REST endpoint.

Svaki PR sa ovakvom izmenom mora biti revidiran od strane bar jednog člana koji nije autor izmene, sa eksplicitnim osvrtom na STRIDE pretnje koje izmena adresira ili uvodi.

8.3 Statička analiza koda samog projekta

Otvorena stavka. Statička analiza *samog* backend koda (Java) trenutno nije pokrenuta u CI-u. Preporuke za implementaciju:

- **SpotBugs** sa *Find Security Bugs* pluginom – gratis, dobro pokriva Java security smell-ove (injection, krypto, hardcoded credentials).
- **SonarQube** ili **SonarCloud** – više funkcija (*code smells*, *coverage*, *security hotspots*), ali traži postavljanje servera ili registraciju cloud naloga.
- **OWASP Dependency-Check** ili **Snyk** – za skeniranje verzija Java zavisnosti (`gradle dependencyCheck`).
- **Trivy** / **Grype** – za skeniranje docker image-a ako se sistem kontejnerizuje.

Konkretno za ovaj sistem, prioritarno bi se uradilo:

1. Pokretanje SpotBugs + Find Security Bugs nad oblak repoom; poređenje nalaza sa STRIDE tabelom (`HARD_CODE_PASSWORD` bi sigurno javio za I3).
2. Pokretanje `dependencyCheck` – ovo otkriva ranjive verzije `jjwt`-a, `MinIO` klijenta, `Spring`-a, itd.

3. `bandit` već integrisan za korisnički kod; može se primeniti i nad `cdk-cli` samim (Python).
4. Dodavanje GitHub Actions / GitLab CI *workflow*-a koji pokreće analize na svaki PR.

9 Otvorene stavke – sumarno

Konsolidovan spisak svih otvorenih stavki, sa kratkim opisom šta bi trebalo uraditi. Stavke su rangirane po prioritetu.

Table 3: Otvorene stavke i preporučeni rad.

ID	Stavka	Prioritet	Akcija
I3	Hardkodovan JWT secret i DB lozinka u <code>application.properties</code>	Visok	Premestiti u env varijable; rotirati ključ pre bilo kakvog deljenja repoa
R1	Nedostaje strukturisan <i>audit log</i>	Visok	Implementirati <code>AuditLog</code> entitet i ulaze u <code>AuthService</code> , <code>UploadService</code> , <code>ExecutionService</code> (videti sekciju 7)
S2	HTTPS još nije konfigurisan	Visok	Konfigurisati TLS na Spring Boot-u, odbijati HTTP <i>schema</i> u CLI-u za ne-produvene serverske URL-ove
SR-15	Rootfs nije <i>read-only</i>	Srednji	<code>is_read_only=true</code> u <code>buildDrive</code> pozivu za rootfs
SR-16	Firecracker se pokreće bez <code>jailer-a</code>	Srednji	Uvesti <code>jailer</code> u komandu u <code>FirecrackerOrchestrator-u</code> ; pokrenuti server kao ne-root
D4	Nema <i>rate limiting</i> -a na <code>/auth</code>	Srednji	<code>Bucket4j</code> filter; <i>lockout</i> posle N neuspelih login-a u Y minuta
D3	Deljen <code>temp/</code> direktorijum	Srednji	<code>Files.createTempDirectory</code> po upload-u, sa <code>cleanup-om</code> u <code>finally</code>
T3	Nema hash-a ext4 image-a	Srednji	Sačuvati SHA-256 u <code>ProjectMetadata-i</code> ; verifikovati pri preuzimanju
S1	Dug <i>access</i> token TTL	Srednji	Skratiti na 15–30 min; uvesti <code>jti</code> black-list za revoke
I4	Host putanje u API odgovoru	Nizak	U <code>ExecutionResult-u</code> vraćati samo <code>programStdout/Stderr</code>
I5	H2 konzola otvorena u <code>SecurityConfig-u</code>	Nizak	Ukloniti pravilo (PostgreSQL je u upotrebi)

ID	Stavka	Prioritet	Akcija
T2	Vrednosti u <i>boot args</i> -u nisu eksplicitno whitelisted	Nizak	Validirati <code>rootFile/workingDirectory</code> regex-om
E2	<code>bash -lc</code> sa <code>String.format</code>	Nizak	Pozvati <code>firecracker</code> direktno kao argumente <code>ProcessBuilder</code> -a
–	Statička analiza Java koda u CI-u	Srednji	Dodati SpotBugs + Find Security Bugs i Dependency-Check u Gradle/CI
R2	Logovi se mogu menjati lokalno	Nizak	Za produkciju: centralizovani logging

10 Zaključak

Sistem *Oblak* pokazuje pažljiv pristup centralnoj pretnji – izvršavanju proizvoljnog korisničkog koda. Izbor Firecracker-a (umesto Docker-a, čistog `subprocess`-a ili Python `exec`-a) je tehnički najjača odluka u arhitekturi i jedina koja realno obezbeđuje izolaciju vrednu tog imena. Višeslojni pristup – ZIP validacija, statička analiza pre izvršavanja, virtualizacija bez mreže, ograničeni resursi, dvostruki timeout – predstavlja zdrav primer *defense in depth*-a.

Najveće slabe tačke u trenutnoj implementaciji nisu eksplicitno tehnološki problemi, već *raniya faza zrelosti*: kredencijali u repozitorijumu (I3), nedostajanje TLS-a (S2), nedostajanje *audit* traga (R1), i nedostajanje *rate limit*-a (D4). Svaki od njih je relativno jeftin za adresiranje i preporučuje se pre bilo kakve produktivne upotrebe sistema.

U pogledu izvršavanja u VM-u (sekcija 6), preporučeno je nekoliko jačanja: `is_read_only=true` za rootfs, uvod `jailer`-a, eksplicitno proveravanje *seccomp* podešavanja. Svaka od ovih izmena malo doprinosi smanjenju ostatak-rizika, koji je u trenutnoj postavci već zadovoljavajuće nizak za akademski prototip.

A Mapiranje koda na bezbednosne mehanizme

Za sledljivost, ovde se daje tabela koja povezuje konkretne klase u kodu sa STRIDE pretnjama koje adresiraju.

B Reference

Sve reference su uvodne, pristupačne tipa "pročitati pre dubljeg ulaska u temu":

Table 4: Klase / fajlovi i pretnje koje adresiraju.

Klasa / fajl	Adresira
security/SecurityConfig.java	SR-1, SR-2 (stateless, BCrypt)
security/JwtService.java	S1 (TTL, signed JWT)
security/JwtAuthFilter.java	SR-1 (filter na svim /api/**)
security/RefreshTokenService.java	S1 (revocation)
security/PasswordPolicyService.java	I2 (policy + common passwords)
core/validation/ZipValidation.java	T1, D1 (zip slip, zip bomb)
core/upload/ZipExtractionService.java	T1 (normalize)
core/analysis/CodeAnalysisService.java	E1 (sloj 2 – statička analiza)
core/analysis/BanditService.java	E1
core/analysis/AntivirusService.java	E1
core/analysis/DependencyAnalysisService.java	E1
firecracker/assets/FirecrackerConfigBuilder.java	D3, E1 (RAM/vCPU; bez mreže)
core/execution/FirecrackerOrchestrator.java	D2, E1 (timeout)
core/execution/ExecutionService.java	E3, I1 (vlasništvo, 404)
firecracker/scripts/agent-runner.sh	D2 (timeout 10s)
cdk-cli/main.py	T4 (chmod 0600)

- Microsoft. *The STRIDE Threat Model*. Microsoft Learn.
- Shostack, A. (2014). *Threat Modeling: Designing for Security*. Wiley.
- OWASP. *Application Threat Modeling*. OWASP Foundation.
- OWASP. *Top Ten* (poslednje izdanje). OWASP Foundation.
- Firecracker. *Production Host Setup i Threat Model*. [firecracker-microvm.github.io](https://github.com/firecracker-microvm/firecracker-microvm.github.io).
- NIST SP 800-63B. *Digital Identity Guidelines: Authentication and Lifecycle Management*.
- NIST SP 800-92. *Guide to Computer Security Log Management*.